



**Università
degli Studi
di Palermo**



Uso delle funzioni in C

CALCOLATORI ELETTRONICI – FONDAMENTI DI PROGRAMMAZIONE
a.a. 2025/2026

Prof. Roberto Pirrone

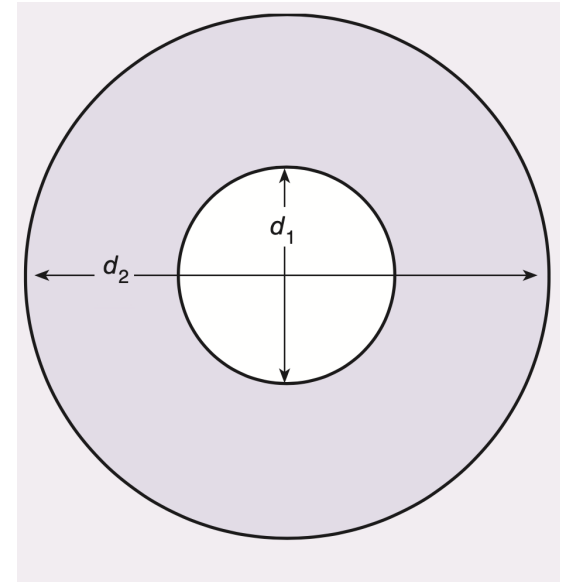


Decomposizione in sotto-problemi

- La scrittura di un programma complesso si articola sempre in sotto-problemi
 - È disponibile del codice scritto da altri
 - Si utilizzano delle API che forniscono delle funzionalità parziali
 - Si effettua una esplicita decomposizione in sotto-problemi per non curarsi inizialmente del dettaglio e definire l'algoritmo complessivo

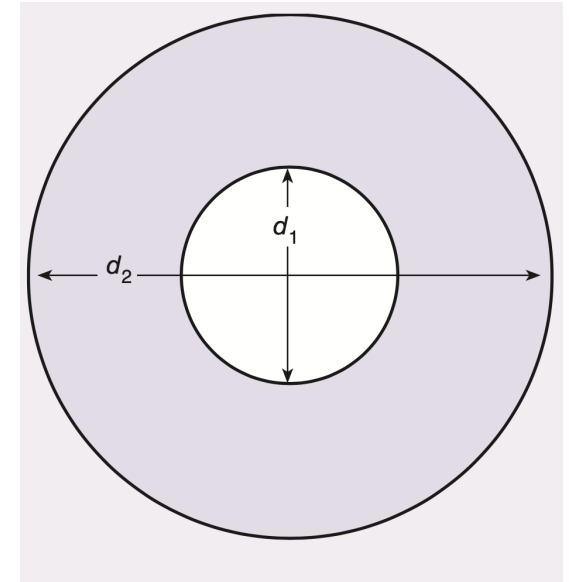
Esempio: Calcolo del peso di un lotto di guarnizioni

- Input:
 - Diametro del buco d_1
 - Diametro del bordo d_2
 - Spessore della guarnizione $spess$
 - Peso soecifico delle guarnizioni p_spec
 - Numero di guarnizioni n
- Output:
 - Peso del lotto $peso$
- Valori costanti:
 - π valore di π (approssimiamo con 3.14156539)



Esempio: Calcolo del peso di una lotto di guarnizioni

1. Acquisisci le dimensioni della guarnizione: d_1 , d_2 , $spess$
2. Acquisisci il numero ed il peso specifico delle guarnizioni: n , p_{spec}
3. Calcola *l'area dell'anello* della guarnizione
4. Calcola il *volume* di una guarnizione
5. Calcola il peso del lotto
6. FINE

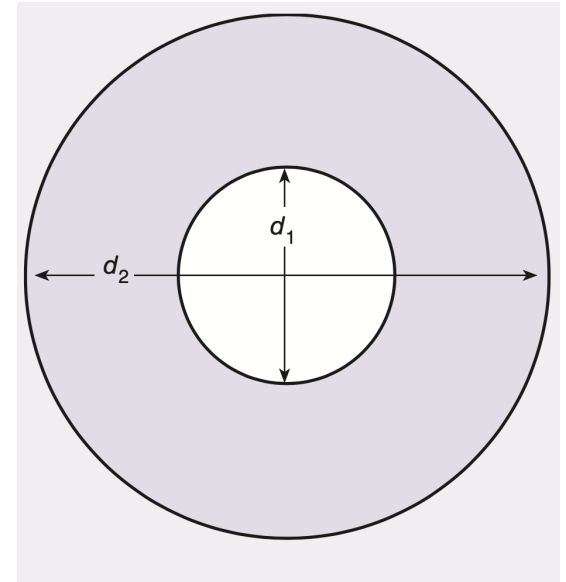


Esempio: Calcolo del peso di una lotto di guarnizioni

- Raffinamento passo 3:

- Input:
 - Diametro del buco d_1
 - Diametro del bordo d_2
- Output:
 - Area dell'anello a_area

1. Calcola *l'area del cerchio* interno
2. Calcola *l'area del cerchio* esterno
3. a_area è pari alla differenza tra le due aree



Esempio: Calcolo del peso di una lotto di guarnizioni

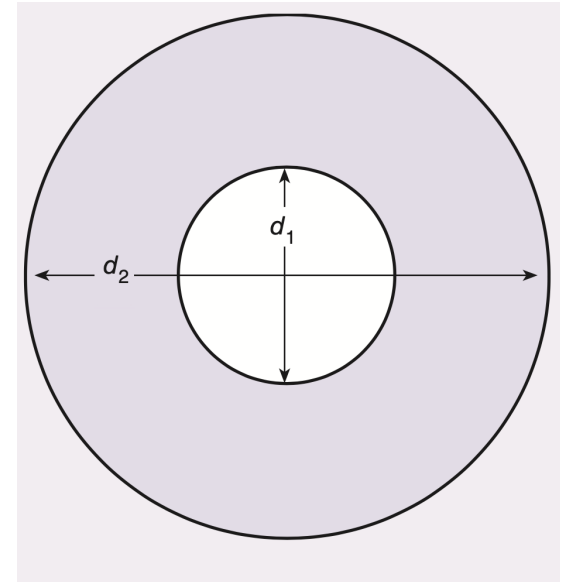
- Raffinamento passo 4:

- Input:
- Spessore della guarnizione $spess$
- Area dell'anello a_area

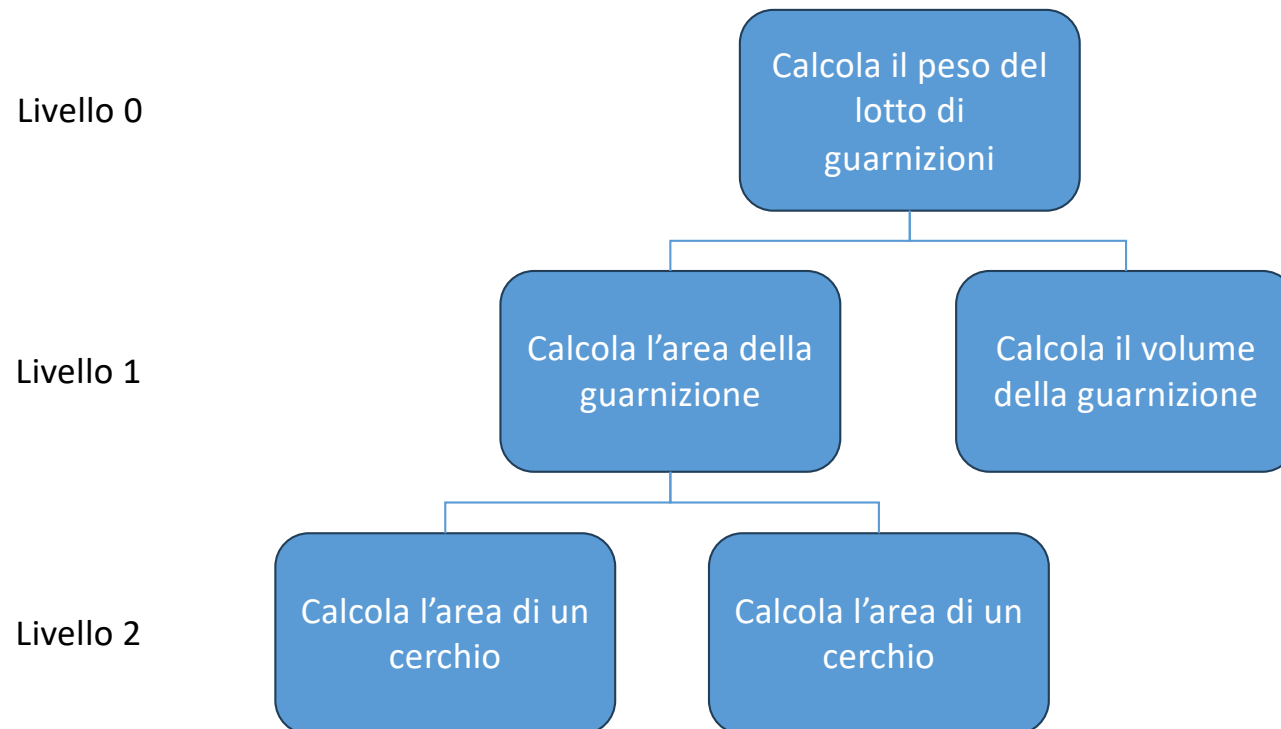
*Il passo 4 ha bisogno
del risultato del passo 3 !!*

- Output:
- Volume dell'anello a_volume

1. a_volume è pari a $a_area * spess$



Diagrammi di struttura



Università
degli Studi
di Palermo



dipartimento
di ingegneria
unipa



Dichiarazione di una funzione

```
SYNTAX:  function interface comment
         ftype
         fname ( formal parameter declaration list )
         {
             local variable declarations
             executable statements
         }
```



Dichiarazione di una funzione

```
/*  
    area_anello: calcola l'area di una rondella circolare come differenza  
    delle due aree dei cerchi di bordo e del buco centrale  
  
    Args:  
        double d1: il diametro interno  
        double d2: il diametro esterno  
  
    Returns:  
        double a_area: l'area risultante  
  
    Pre: i valori d1 e d2 devono essere positivi e  $d2 > d1$   
    Post: nessuna  
*/  
double area_anello(double d1, double d2){  
  
    double a_area;  
  
    a_area = (PI/4)*(d2*d2 - d1*d1);  
    return a_area  
}
```

Precondizione: ciò che deve essere vero
prima della chiamata della funzione

Postcondizione: ciò che deve essere vero
dopo la chiamata della funzione

Prototipo di una funzione

```
/*  
    area_anello: calcola l'area di una rondella circolare come differenza  
    delle due aree dei cerchi di bordo e del buco centrale
```

Args:

```
    double d1: il diametro interno  
    double d2: il diametro esterno
```

Returns:

```
    double a_area: l'area risultante
```

Pre: i valori d1 e d2 devono essere positivi e $d2 > d1$

Post: nessuna

```
*/  
double area_anello(double d1, double d2);  
  
int main(int argc, char *argv[]){  
    ...  
}
```

La dichiarazione del prototipo mi consente di non pensare a come implementare la funzione mentre sono impegnato con la codifica dell'algoritmo principale !!

```
double area_anello(double d1, double d2){  
  
    double a_area;  
  
    a_area = (PI/4)*(d2*d2 - d1*d1);  
    return a_area  
}
```



Università
degli Studi
di Palermo



dipartimento
di ingegneria
unipa



Parametri di una funzione

/*

area_anello: calcola l'area di una rondella circolare come differenza delle due aree dei cerchi di bordo e del buco centrale

Args:

double d1: il diametro interno
double d2: il diametro esterno

Returns:

double a_area: l'area risultante

Pre: i valori d1 e d2 devono essere positivi e $d2 > d1$

Post: nessuna

*/

double area_anello(double d1, double d2);

```
int main(int argc, char *argv[]){  
    double diametro_int, diametro_est, area;  
    ...  
    area = area_anello(diametro_int, diametro_est);  
    ...  
}
```

Parametri attuali

*I valori dei parametri attuali vengono **copiati** all'interno dei corrispondenti parametri formali i quali poi vengono usati come le altre variabili locali della funzione*

Parametri formali

```
double area_anello(double d1, double d2){  
    double a_area;  
    a_area = (PI/4)*(d2*d2 - d1*d1);  
    return a_area;  
}
```

Passaggio dei parametri ad una funzione per riferimento

- Supponiamo di avere una funzione che converta un carattere da minuscolo in maiuscolo
- Voglio che la funzione modifichi *direttamente* il carattere

```
void maiusc(char c){  
    if(c >= 97 && c <= 122){    // Codici ASCII  
                                // delle minuscole  
        c -= 32;                // mi sposto al codice ASCII  
                                // del corrispettivo maiuscolo  
    }  
    return;  
}
```

Void è il tipo nullo usato per indicare che non c'è un risultato esplicito da restituire

Passaggio dei parametri ad una funzione per riferimento

- Supponiamo di avere una funzione che converta un carattere da minuscolo in maiuscolo
- Voglio che la funzione modifichi *direttamente* il carattere

```
void maiusc(char c){  
    if(c >= 97 && c <= 122){    // Codici ASCII  
                                // delle minuscole  
        c -= 32;                // mi sposto al codice ASCII  
                                // del corrispettivo maiuscolo  
    }  
    return;  
}
```

Se usiamo questa funzione non converte in maiuscolo!! Perché??



Università
degli Studi
di Palermo



dipartimento
di ingegneria
unipa



Passaggio dei parametri ad una funzione per riferimento

- Supponiamo di avere una funzione che converta un carattere da minuscolo in maiuscolo
- Voglio che la funzione modifichi *direttamente* il carattere

```
void maiusc(char c){  
    if(c >= 97 && c <= 122){    // Codici ASCII  
                                // delle minuscole  
        c -= 32;                // mi sposto al codice ASCII  
                                // del corrispettivo maiuscolo  
    }  
    return;  
}
```

Il parametro formale viene eliminato alla fine dell'esecuzione e non ce n'è traccia nel main

Passaggio dei parametri ad una funzione per riferimento

- Supponiamo di avere una funzione che converta un carattere da minuscolo in maiuscolo
- Voglio che la funzione modifichi *direttamente* il carattere

```
void maiusc(char *c){  
    if(*c >= 97 && *c <= 122){ // Codici ASCII  
                                // delle minuscole  
        *c -= 32; // mi sposto al codice ASCII  
                // del corrispettivo maiuscolo  
    }  
    return;  
}
```

Come fare?? Usiamo i puntatori!! Facciamo riferimento diretto alla locazione di memoria del parametro attuale e quindi salviamo le modifiche